



AI-Based Security Orchestration, Automation, and Response (SOAR) Tool

Hizam Mohammed Ali Al-Shajara	202210102478
Hamoud Abdullah Saleh Abu Amrah	202310101609
Mohammed Hameed Qasim Mohammed	202310100174
Mohammed Taha Qasim Al-Warafi	202310100461
Marwan Mohammed Saeed Al-Ameer	202310100177
Noah Ahmed Mohammed Maraq	202310100452

Supervised By

Dr. Raed Saeed

A graduation project submitted to the Department of Computer Science in partial fulfillment
of the requirements for the Bachelor's Degree in Cybersecurity and Networking

Abstract

Modern Security Operations Centers (SOCs) face increasing challenges in managing large volumes of security alerts, false positives, and heterogeneous security data. These conditions place significant pressure on security analysts, increase manual triage effort, and may delay the identification and response to high-priority incidents. Although Security Orchestration, Automation, and Response (SOAR) platforms can automate repetitive security operations, conventional workflows often rely on predefined rules and static playbooks that may provide limited adaptability when alert context and risk conditions change.

This project proposes an AI-Based Security Orchestration, Automation, and Response (SOAR) tool designed to support intelligent security alert analysis and more adaptive incident handling. The proposed tool integrates artificial intelligence and machine learning techniques with SOAR capabilities to analyze incoming security alerts, assess contextual risk, and dynamically prioritize incidents before initiating appropriate response workflows. The architecture incorporates major functional components for alert ingestion, context enrichment, AI/ML-based analysis and risk scoring, prioritization, and automated response orchestration.

The project adopts a tailored Design Science Research methodology that combines problem investigation, literature review, architectural design, machine learning development, system integration, and evaluation. The current stage focuses on identifying operational challenges, reviewing existing approaches, establishing the research gap, and designing the proposed system architecture. Subsequent development stages will involve data preparation, implementation of the AI/ML analysis mechanism, integration with SOAR workflows, and prototype evaluation using appropriate machine-learning and operational metrics.

Ultimately, the proposed tool aims to investigate how intelligent alert analysis and dynamic prioritization can be integrated with automated security response processes. The project is expected to provide a foundation for reducing unnecessary manual triage effort, supporting context-aware response decisions, and improving the coordination between alert analysis and incident response within SOC environments.

Authorization

We authorize the University of Science and Technology, Faculty of Computer Science and Technology, to supply copies of our graduation project report to libraries, organizations, or individuals upon request.

Student Name	Student No.	Signature
Hizam Mohammed Ali Al-Shajara	202210102478	
Hamoud Abdullah Saleh Abu Amrah	202310101609	
Mohammed Hameed Qasim Mohammed	202310100174	
Mohammed Taha Qasim Al-Warafi	202310100461	
Marwan Mohammed Saeed Mahdi Al-Ameer	202310100177	
Noah Ahmed Mohammed Maraq	202310100452	

Date: ____ / ____ / ____

Dedication

First and foremost, all praise and gratitude are due to Almighty Allah for granting us the strength, patience, knowledge, and determination to complete this work.

We dedicate this project to our beloved parents, whose continuous prayers, sacrifices, support, and encouragement have been the foundation of our educational journey. Their belief in us has given us the motivation to overcome challenges and continue striving toward our goals.

We also dedicate this work to our families, brothers, sisters, and loved ones who have supported us throughout our years of study and stood beside us during the demanding stages of this project.

To our friends and colleagues, who shared with us the challenges, experiences, knowledge, and memorable moments of our academic journey, we extend our sincere appreciation.

Finally, we dedicate this humble work to everyone who supported us, encouraged us, shared their knowledge, or contributed in any way to making this project possible.

To all of you, we dedicate this work with sincere gratitude and appreciation.

Acknowledgment

Before and above all, we express our deepest gratitude to **Almighty Allah** for His blessings, guidance, and strength, which enabled us to complete this project.

We would like to express our sincere gratitude and appreciation to **Dr. Raed Saeed**, our supervisor, for his valuable guidance, scientific advice, continuous support, constructive feedback, and encouragement throughout the development of this project.

We are also deeply grateful to our families for their constant support, patience, motivation, and encouragement throughout our academic journey. Their belief in us has been a continuous source of strength and inspiration.

Finally, we extend our sincere appreciation to our friends, colleagues, and everyone who contributed to this project directly or indirectly. Their support and assistance were invaluable to the completion of this work.

Supervisor Certification

We certify that the preparation of this project entitled “**AI-Based Security Orchestration, Automation, and Response (SOAR)**”, prepared by **Hizam Al-Shajara, Hamoud Abu Amrah, Mohammed Mohammed, Mohammed Al-Warafi, Marwan Al-Ameer, and Noah Maraq**, was made under my supervision at the **Computer Science Department**, in partial fulfillment of the requirements for the **Bachelor’s Degree in Cybersecurity and Networking**.

Supervisor Name: Dr. Raed Saeed

Signature: _____

Date: ____ / ____ / ____

Examination Committee

Project Titel: AI-Based Security Orchestration, Automation, and Response (SOAR)

Supervisor

No.	Name	Position	Signature
1	Dr. Raed Saeed	Faculty Member	

Examination Committee

No.	Name	Position	Signature
1			
2			
3			

Head of Department

Dr. Nabeel AL-Mekhlafy

Table of Contents

Abstract	I
Authorization	II
Dedication.....	III
Acknowledgment.....	IV
Supervisor Certification	V
Examination Committee	VI
List of Figures	IX
List of Tables	IX
List of Abbreviations	IX
<i>Chapter 1:</i>	11
Introduction and Problem Statement	11
1.1 Overview of Modern SOC's and SOAR Platforms.....	12
1.2 The Problem Statement (Alert Fatigue and Static Triage)	13
1.3 Motivation and Project Significance	14
1.4 Project Objectives 1.4.1 Primary Objective	15
1.4.2 Specific Objectives	15
1.5 Project Methodology	15
1.6 Project Scope and Limitations.....	16
1.6.1 Project Scope	16
1.6.2 Project Limitations.....	16
1.7 Report Organization.....	17
Chapter 1: Introduction and Problem Statement.....	17
Chapter 2: Background and Literature Review:	17
Chapter 3: Proposed System Architecture	17
Chapter 4: Research Methodology and Model Development	17
Chapter 5: Preliminary Implementation and Initial Results	17
Chapter 6: Expected Outcomes and Project Timeline	17
<i>Chapter 2</i>	18
Background and Literature Review	18
2.1 Introduction	19

2.2 Technical Background: SOC an Conventional SOAR	19
2.2.1 SOC Operational Workflow and SIEM.....	19
2.2.2 Conventional SOAR, Playbooks, and API-Based Orchestration.....	20
2.2.3 Limitations of Conventional SOAR Automation	21
2.3 AI and ML as a Force Multiplier in SOC	21
2.3.1 AI/ML Capabilities in Security Operations	21
2.3.2 Human-AI Teaming and Controlled Automation	22
2.4 Review of Existing ML Triage Systems	23
2.4.1 Supervised Event Classification and Actionability Assessment.....	23
2.4.2 Context-Aware Alert Triage and Prioritization	24
2.4.3 Adaptive and Resource-Aware Prioritization	25
2.4.4 Comparative Analysis of Existing Approaches.....	25
2.5 Gap Analysis and Problem Synthesis	27
Chapter 3:	30
Proposed System Architecture	30
3.1 Introduction	31
3.2 High-Level Architecture Overview.....	31
3.2.1 Agent Layer Overview	33
3.3 Ingestion and Connectors Module	34
3.5 Machine Learning Triage and Scoring Engine	37
3.7 Automated Response Policy	40
3.8 Analyst Feedback Loop.....	42
References	44

List of Figures

Figure 3-1: High-Level Architecture of the Proposed AI-Based SOAR Tool

Figure 3-2: Ingestion and Context Enrichment Sub-Module Architecture

Figure 3-3: Context Enrichment Data Parsing Pipeline

Figure 3-4: Machine Learning Triage and Risk Scoring Architecture

Figure 3-5: Alert Prioritization, Routing, and Automated Response Pipeline

Figure 3-6: Automated Response Policy Evaluation and SOAR Workflow Pipeline

Figure 3-7: Analyst Feedback, Policy Adjustment, and Periodic Model Refinement Loop

List of Tables

Table 2-1: Comparative Analysis of Existing ML-Based Alert Triage and Prioritization Systems

List of Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
API	Application Programming Interface
DNN	Deep Neural Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
DSR	Design Science Research
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
MAS	Multi-Agent System
MDR	Managed Detection and Response
ML	Machine Learning
MTTR	Mean Time to Respond
NN	Neural Network
RF	Random Forest
RL	Reinforcement Learning
SAC	Soft Actor-Critic
SIEM	Security Information and Event Management

SLA	Service Level Agreement
SOAR	Security Orchestration, Automation, and Response
SOC	Security Operations Center
TD3	Twin Delayed Deep Deterministic Policy Gradient
TI	Threat Intelligence
TTPs	Tactics, Techniques, and Procedures

Chapter 1:
Introduction and Problem Statement

1.1 Overview of Modern SOC and SOAR Platforms

A Security Operations Center (SOC) is a centralized operational function responsible for monitoring, detecting, investigating, and responding to organizational cyber threats (Khayat et al., 2025). To protect corporate networks, modern SOC deploy a diverse array of security sensors across network and host layers (Kinyua & Awuah, 2021). To manage the resulting data, organizations implement Security Information and Event Management (SIEM) platforms, which provide centralized log management, security-event collection, and basic correlation (Bridges et al., 2023; Khayat et al., 2025). However, security monitoring inherently relies on the diverse outputs produced by multi-vendor security stacks. Because these tools operate with distinct log schemas, vendor-specific data representations, and unique alert mechanisms, the resulting security data is highly heterogeneous and complex to analyze (Kinyua & Awuah, 2021).

Within the SOC, security analysts are structured into a tiered organizational model to monitor security feeds and manage incidents (Bridges et al., 2023). Junior analysts conduct initial alert triage, following standard operating procedures to evaluate whether an incoming alert represents a legitimate threat or benign activity (Bridges et al., 2023). When an alert requires investigation, analysts must manually gather contextual evidence from multiple disparate security sources (Khayat et al., 2025). This manual investigation process introduces a phenomenon known as context switching, where analysts repeatedly navigate across different application interfaces, hold investigative hypotheses in memory, and manually correlate heterogeneous data sources (Bridges et al., 2023). Consequently, pivoting between isolated security consoles to execute repetitive diagnostic steps creates significant operational complexity (Kinyua & Awuah, 2021).

To address the operational friction of managing fragmented toolstacks, Security Orchestration, Automation, and Response (SOAR) emerged as a critical architectural paradigm (Kinyua & Awuah, 2021). SOAR refers to software platforms designed to integrate disparate security applications and human processes into a unified framework (Kinyua & Awuah, 2021). While SIEM systems focus on log management and threat identification, SOAR complements them by extending the incident response lifecycle to support automated containment, eradication, and recovery (Karlzén & Sommestad, 2023). By automating routine diagnostic tasks and orchestrating security tools at the application programming interface (API) layer, SOAR accelerates response operations through predefined checklists known as playbooks or workflows (Bridges et al., 2023; Ismail et al., 2025). Rather than replacing personnel, SOAR acts as a force multiplier that empowers human analysts by standardizing procedures while keeping them involved in critical oversight (Bridges et al., 2023; Kinyua & Awuah, 2021).

A representative conventional SOAR workflow progresses through a structured sequence:

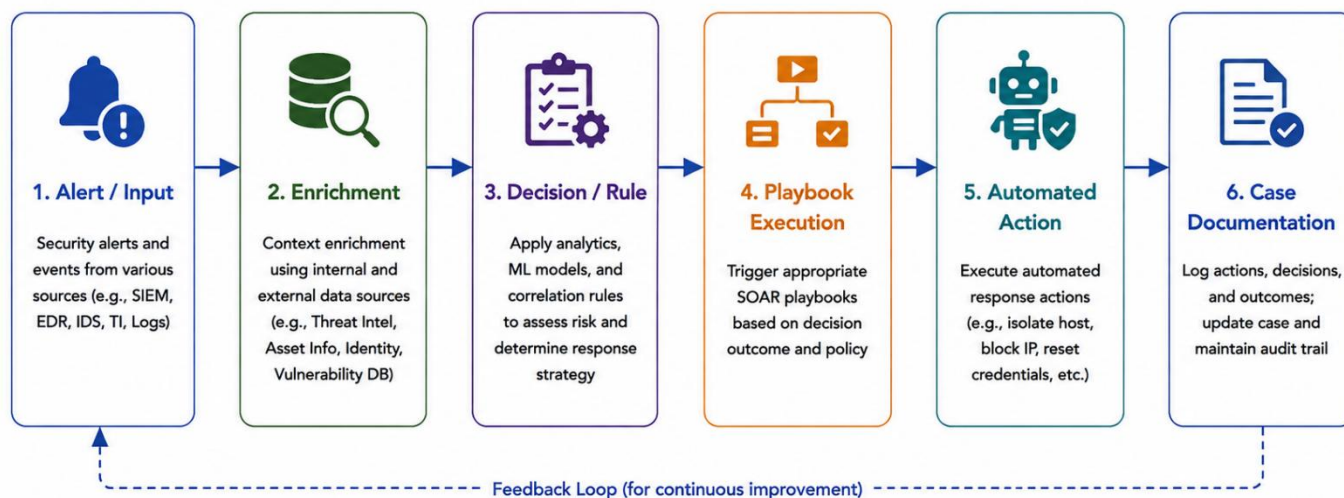


Figure 1-1: End-to-End SOAR Workflow from Alert to Automated Response

Alert/Input → Enrichment → Decision/Rule → Playbook Execution → Automated Action → Case Documentation (Karlzén & Sommestad, 2023). The process begins when the platform ingests a raw security alert from a detection sensor (Zhang et al., 2025). In the enrichment stage, the system automatically executes diagnostic queries to gather additional context, such as retrieving user attributes or querying external reputation databases (Bridges et al., 2023). This accumulated evidence is evaluated against static threshold rules during the decision stage (Zhang et al., 2025). If the criteria are met, the playbook execution stage activates a pre-codified sequence of steps (Ismail et al., 2025), leading to an automated action where technical countermeasures are executed directly on targeted infrastructure (Agrawal et al., 2026). Finally, the case documentation stage logs the execution outcomes and updates incident tickets for analyst review (Bridges et al., 2023). Although SOAR automation can reduce repetitive manual effort, its effectiveness depends in part on the volume, quality, and contextual clarity of incoming security alerts.

1.2 The Problem Statement (Alert Fatigue and Static Triage)

Modern Security Operations Centers (SOCs) face an unsustainable influx of security data, accumulating up to 100 gigabytes of log and alert data daily. A substantial portion of security alerts may consist of false positives and non-actionable noise. Processing this low-fidelity noise demands significant operational effort and detracts from proactive threat hunting.

This convergence of high alert volumes and excessive false positives causes high analyst workload and alert fatigue. In conventional workflows, analysts manually investigate alerts through continuous context switching—repeatedly pivoting across disparate application windows to gather evidence. This repetitive manual triage creates cognitive overload, contributing to acute stress and analyst burnout.

To alleviate these burdens, organizations implement automated workflows; however, reliance on predefined rules and static playbooks may limit adaptability. While conventional automation efficiently executes repetitive steps, deterministic logic can constrain effectiveness when alert context or risk conditions change dynamically. Rigid conditional statements may become less effective against novel or evasive threats that fall outside anticipated patterns.

Beyond static logic, the core operational challenge is prioritization. When prioritization relies primarily on fixed decision rules, highly critical incidents can become buried beneath lower-value noise. Unprioritized queues delay the handling of actionable threats, whereas dynamic, risk-aware ordering can reduce the time critical incidents spend waiting in analyst queues by 22.9%. Ultimately, these challenges motivate the ongoing need to investigate how intelligent alert analysis and prioritization can be integrated with operational workflows to support adaptive incident handling.

1.3 Motivation and Project Significance

The operational challenges of alert fatigue, cognitive overload, and excessive false positives establish a critical need for a more intelligent approach to security alert handling. As modern Security Operations Centers (SOCs) navigate increasingly complex network environments, traditional manual triage processes struggle to scale effectively. Security teams require advanced mechanisms capable of distinguishing high-value, actionable incidents from low-value, benign security noise. Consequently, there is a fundamental operational motivation to improve triage efficiency, ensuring that genuine threats are identified rapidly without simultaneously increasing the manual investigative burden placed on human analysts.

From a technological perspective, this project is motivated by the complementary capabilities of artificial intelligence and existing security platforms. Artificial Intelligence and Machine Learning (AI/ML) provide data-driven capabilities for intelligent alert analysis, classification, contextual assessment, and decision support. Conversely, Security Orchestration, Automation, and Response (SOAR) platforms specialize in deterministic execution, providing the infrastructure for multi-tool orchestration, workflow automation, playbook execution, and rapid response deployment. Integrating these distinct domains provides an opportunity to connect intelligent, context-aware analysis directly with automated execution pipelines, extending the analytical capabilities available within conventional SOAR operations.

Academically, this study aims to investigate an integration challenge identified in the reviewed cybersecurity literature. Existing research demonstrates the use of machine learning for security alert analysis and the use of SOAR workflows for automated incident response, while the integration between these capabilities remains an area worthy of further investigation. This project therefore investigates the architectural integration of AI/ML-based alert analysis, risk-based scoring, dynamic prioritization, and SOAR automated response. Specifically, the proposed tool seeks to examine how intelligent triage outputs can support more adaptive response decisions within operational SOAR workflows.

Practically, the integration of intelligent prioritization with automated response mechanisms may provide operational value for SOC environments. By applying contextual risk assessment to incoming alerts, the proposed system is intended to reduce unnecessary manual triage effort and minimize distractions caused by low-value alerts. This approach aims to help security analysts focus their time and attention on higher-priority incidents. By improving the coordination between alert analysis and automated incident response, the tool could support more context-aware response decisions and contribute to more efficient SOC operations.

1.4 Project Objectives

1.4.1 Primary Objective

The primary objective of this project is to investigate, design, and propose an integrated AI-Based Security Orchestration, Automation, and Response (SOAR) tool. This tool aims to address operational bottlenecks by integrating intelligent security alert analysis, risk-based scoring, dynamic prioritization, and automated incident response to support more adaptive and efficient security operations.

1.4.2 Specific Objectives The specific objectives of this project are to:

1. **Analyze** the operational challenges of current SOC alert management, specifically including alert fatigue and the limitations of static triage.
2. **Review** existing AI/ML-based approaches and academic literature related to security alert analysis and prioritization.
3. **Design** the proposed AI-Based SOAR architecture and clearly identify its major functional components.
4. **Develop** an AI/ML-based mechanism designed for context-aware alert analysis and risk assessment.
5. **Integrate** the outcomes of dynamic risk-based prioritization with SOAR response decision-making to guide automated workflows.
6. **Define** a structured evaluation approach to assess the proposed tool using appropriate machine-learning and operational metrics.

1.5 Project Methodology

This project adopts a tailored Design Science Research (DSR) methodology, selected because it provides a structured academic framework for designing, developing, and evaluating an IT artifact intended to address a practical cybersecurity problem. The methodology is adapted to the requirements of the AI-Based SOAR project by incorporating the research, architectural design, machine learning development, integration, and evaluation activities required to investigate intelligent security alert management.

The first phase, Problem Identification and Objectives, focuses on defining the operational challenges associated with SOC alert management, including alert fatigue and static triage, and establishing the

objectives of the project. The second phase, Literature Review and Evidence Mapping, examines existing research on SOC alert management, conventional SOAR, AI/ML-based alert analysis, risk scoring, dynamic prioritization, and automated incident response. These activities provide the academic foundation for identifying and validating the research gap. Both phases are undertaken primarily within Graduation Project 1.

The third phase, System Architecture and Component Design, develops the proposed high-level architecture for integrating AI-based alert analysis, risk scoring, dynamic prioritization, and SOAR response workflows. The initial architecture is established during Graduation Project 1 and may be refined during Graduation Project 2 as implementation requirements are identified. The fourth phase, Data Collection and Preparation, and the fifth phase, AI/ML Model Development and Risk Scoring, are planned for Graduation Project 2. These phases will involve preparing security alert data, performing feature engineering, developing the selected AI/ML mechanism, and establishing an appropriate risk-scoring approach.

The sixth phase, SOAR Integration and Workflow Automation, will connect the intelligent analysis and prioritization mechanisms with the proposed SOAR response workflows. The final phase, Prototype Testing and Evaluation, will assess the resulting tool using appropriate machine-learning metrics and operational measures, such as false-positive rates, prioritization performance, and queue-related indicators. Findings from evaluation will be used to support iterative refinement of the proposed artifact and to assess its alignment with the project objectives.

1.6 Project Scope and Limitations

1.6.1 Project Scope

This project encompasses the design and planned development of an integrated AI-Based Security Orchestration, Automation, and Response (SOAR) tool. The primary operational context focuses on Security Operations Center (SOC) alert management. Specifically, the scope encompasses the integration of artificial intelligence and machine learning for security alert analysis, context-aware risk scoring, and dynamic alert prioritization. Furthermore, it includes connecting these analytical components with automated SOAR response workflows and decision-making mechanisms. Implementation and testing are planned to be conducted within a controlled prototype and simulation environment appropriate for a graduation project. The project scope also involves defining appropriate machine-learning and operational metrics to evaluate the tool. Finally, the scope emphasizes architectural integration rather than the invention of new foundational machine-learning algorithms or commercial platforms.

1.6.2 Project Limitations

Several realistic limitations bound this study. First, model training and evaluation may depend on synthetic, public, or otherwise limited security alert datasets, which may not fully replicate the complex telemetry of every enterprise environment. Consequently, machine-learning performance can be influenced

by data quality, distribution changes, and evolving threat behaviors over time. Second, validation is constrained to a laboratory or prototype environment rather than a live, large-scale enterprise SOC, restricting large-scale operational testing. Third, fully autonomous execution of high-impact security actions introduces inherent operational risks; therefore, the tool incorporates human oversight and approval mechanisms for uncertain or potentially disruptive response actions. Finally, project time and resource constraints may restrict deep integration with proprietary commercial security infrastructure and limit testing across legacy systems.

1.7 Report Organization

This report is organized into six chapters that systematically present the research, system architecture, methodology, implementation, and expected outcomes of the AI-Based SOAR project.

Chapter 1: Introduction and Problem Statement

introduces the project context by providing an overview of modern Security Operations Centers and SOAR platforms, formulating the problem statement regarding alert fatigue and static triage, and outlining the project motivation, objectives, methodology, scope, limitations, and report structure.

Chapter 2: Background and Literature Review:

establishes the technical background and literature review, covering SOC operations, conventional SOAR workflows, the role of artificial intelligence and machine learning in SOC operations, existing AI/ML-based alert triage approaches, risk scoring and dynamic prioritization, and the resulting research gap.

Chapter 3: Proposed System Architecture

presents the proposed AI-Based SOAR system architecture, describing its major functional components, including alert ingestion, context enrichment, AI/ML-based triage and scoring, prioritization and suppression logic, automated response policies, and SOAR integration.

Chapter 4: Research Methodology and Model Development

describes the research methodology and model development process, including data collection and preparation, feature engineering, model selection and training, risk-scoring development, threshold tuning, and evaluation metrics.

Chapter 5: Preliminary Implementation and Initial Results

presents the preliminary implementation and initial evaluation activities, including the development environment, initial testing data, model performance analysis, and preliminary operational observations.

Chapter 6: Expected Outcomes and Project Timeline

presents the expected outcomes, project deliverables, foundation for future development in Graduation Project 2, and the overall project timeline.

Chapter 2

Background and Literature Review

2.1 Introduction

Modern Security Operations Centers (SOCs) operate in increasingly complex environments characterized by heterogeneous security technologies, large volumes of security telemetry, and substantial dependence on human analysts for investigation and incident handling. The use of multiple security products from different vendors can create fragmented workflows and inconsistent data representations, increasing the difficulty of maintaining an integrated view of security operations (Kinyua & Awuah, 2021). In response to these challenges, artificial intelligence and machine learning (AI/ML) have been increasingly investigated as mechanisms for augmenting SOC capabilities through automated analysis, threat detection, classification, behavioral analytics, and decision support (Khayat et al., 2025). Recent research has further explored AI-assisted and agent-based approaches for tasks such as alert triage, threat intelligence analysis, incident response, and security reporting, while emphasizing the continued importance of human oversight in operational environments (Srinivas et al., 2025).

This chapter reviews the technical foundations and prior research relevant to the proposed AI-Based SOAR tool. It first examines conventional SOC and SOAR operations, then discusses the role of AI/ML in augmenting security operations. Existing machine-learning-based alert triage and prioritization approaches are subsequently reviewed and compared. Finally, the chapter synthesizes the limitations of existing approaches to establish the research gap addressed by the proposed tool.

2.2 Technical Background: SOC and Conventional SOAR

2.2.1 SOC Operational Workflow and SIEM

A modern Security Operations Center operates as a coordinated environment in which security telemetry is collected from multiple sources, analyzed for indicators of malicious activity, and escalated for investigation when necessary. These sources may include network and endpoint monitoring systems, intrusion detection technologies, firewalls, threat-intelligence feeds, and other security controls. Because individual products frequently use different data representations and operate independently, SOC environments require mechanisms that consolidate security information and provide analysts with a broader operational view (Kinyua & Awuah, 2021).

Security Information and Event Management (SIEM) systems provide an important foundation for this process by aggregating and querying security data from distributed sources. However, the operational role of a SIEM differs from that of a SOAR platform. Bridges et al. (2023) distinguish the two by emphasizing that SIEM primarily supports broad data collection and querying, whereas SOAR introduces configurable workflows that guide or automate investigation and incident-response activities. Consequently, the output of a SIEM may serve as an input to subsequent triage, enrichment, investigation, and response processes rather than representing the complete incident-response lifecycle.

Within this workflow, analysts must determine whether incoming alerts warrant further investigation, collect supporting evidence, and coordinate response activities. SOAR technologies seek to reduce the fragmentation of these activities by combining alert information, threat intelligence, workflow automation, and analyst collaboration within a common operational environment (Bridges et al., 2023). Bridges et al. identify data ingestion, correlation and prioritization, process automation, and analyst collaboration as defining capabilities of SOAR systems. These functions allow security information to move beyond simple collection toward structured operational handling.

2.2.2 Conventional SOAR, Playbooks, and API-Based Orchestration

The core operational mechanism of conventional SOAR is the **playbook or workflow**. A playbook defines a structured sequence of steps and actions used to handle a particular incident type, with execution triggered when predefined conditions or events are satisfied (Kinyua & Awuah, 2021). Such workflows can automate activities including information enrichment, ticket creation, notifications, containment procedures, and interactions with security controls. Runbooks may additionally introduce conditional flow-control mechanisms and points at which an analyst can intervene before execution continues (Kinyua & Awuah, 2021).

The concept of orchestration extends beyond automating a single action. Many incidents require coordinated operations across multiple security technologies. For example, responding to a phishing incident may involve inspecting a suspicious file, querying external reputation information, blocking a sender, and modifying a firewall policy. Karlzén and Sommestad (2023) note that pre-prepared playbooks provide a mechanism for coordinating such defensive actions, while their analysis of commercial SOAR repositories shows that practical playbooks perform both detection-oriented enrichment and concrete response operations. Their study analyzed 609 publicly available playbooks from Rapid7, Microsoft Sentinel, ThreatConnect, and Splunk SOAR, demonstrating the breadth of actions represented in operational SOAR ecosystems (Karlzén & Sommestad, 2023).

At the implementation level, orchestration depends heavily on **Application Programming Interfaces (APIs)**. Security tools expose APIs through which a SOAR platform can query information or initiate actions on external technologies. This creates a logical relationship in which an incident-response task is translated into an operation supported by a specific security product. Sworna et al. (2023), for example, describe response tasks in Cortex SOAR that require APIs from technologies such as VirusTotal, CrowdStrike, and Carbon Black Response. Their work highlights that tool and API selection is an important technical component of multi-vendor SOAR orchestration, particularly because different products expose different interfaces, documentation structures, parameters, and capabilities.

2.2.3 Limitations of Conventional SOAR Automation

Although conventional SOAR improves coordination and reduces repetitive manual activity, automation does not eliminate operational complexity. Playbooks must be designed, configured, maintained, and integrated with the security technologies available in a particular environment. Bridges et al. (2023) note that SOAR adoption can involve substantial configuration effort, organizational process changes, training requirements, and technical integration costs. Furthermore, heterogeneous security tools can complicate orchestration because response tasks may require SOC personnel to identify and maintain appropriate vendor-specific APIs (Sworna et al., 2023).

Conventional workflows are also strongly dependent on the logic encoded into their playbooks. Predefined sequences are well suited to repeatable procedures when the necessary conditions and actions are known in advance, but their adaptability is constrained by the information and decision logic incorporated during workflow design (Ismail et al., 2025). Consequently, conventional orchestration provides a structured execution layer but does not, by itself, provide intelligent interpretation of changing alert context or adaptive alert prioritization. This limitation motivates the use of AI/ML as an analytical and decision-support layer that can complement conventional SOAR automation, as examined in the following section.

2.3 AI and ML as a Force Multiplier in SOC

2.3.1 AI/ML Capabilities in Security Operations

The integration of artificial intelligence and machine learning (AI/ML) into security operations extends the capabilities of conventional rule-based monitoring and automation. While traditional security workflows are generally designed around predefined conditions and known response procedures, AI/ML techniques can analyze large and heterogeneous security datasets to identify patterns, classify events, detect anomalous behavior, and support security decision making (Khayat et al., 2025). Khayat et al. (2025) identify applications of AI/ML across several SOC functions, including threat detection and classification, behavioral analytics, threat intelligence, and automated incident response. In this role, AI/ML does not necessarily replace existing SOC technologies; rather, it can provide an analytical layer that complements the information collected by SIEM and the workflow execution capabilities provided by SOAR.

This relationship has been described as a **force-multiplier effect** for SOC analysts. Kinyua and Awuah (2021) argue that AI/ML can extend SOAR by improving analytical, threat-intelligence, detection, and response capabilities while orchestration technologies continue to coordinate operational actions. From this perspective, the complementary roles of the technologies can be distinguished: SOAR provides workflow management, integration, and execution, whereas AI/ML can contribute data-driven inference to activities that are difficult to represent entirely through manually specified rules. This distinction is

particularly relevant to alert handling, where the significance of an event may depend on multiple attributes and surrounding contextual information rather than on a single static severity value.

The range of AI techniques applicable to SOC operations has also expanded. Supervised and unsupervised machine-learning methods can support functions such as event classification, anomaly detection, and risk assessment, while deep-learning and reinforcement-learning approaches can support more complex pattern recognition and adaptive decision processes (Khayat et al., 2025). More recently, Large Language Models (LLMs) and AI agents have been investigated for functions such as log summarization, alert triage, threat-intelligence analysis, incident response, and report generation (Srinivas et al., 2025). This broader use of AI indicates a shift from applying intelligence primarily to isolated detection tasks toward incorporating AI into multiple stages of SOC workflows. However, the suitability of a particular technique depends on factors such as data availability, interpretability, computational requirements, operational risk, and the degree of autonomy permitted within the environment.

2.3.2 Human-AI Teaming and Controlled Automation

Increasing AI capability does not imply that all SOC decisions should be fully automated. Security operations include actions with substantially different levels of uncertainty and operational impact. A low-risk enrichment query, for example, does not carry the same consequences as disabling an account, isolating a production endpoint, or modifying a network-control policy. Consequently, the relationship between AI recommendations and human authority becomes an important design consideration when intelligent analysis is connected to automated response.

Chhetri et al. (2024) propose the A2C framework for human-AI teaming in SOC environments, emphasizing **flexible autonomy** rather than a fixed division between manual and automated operation. Their framework distinguishes among automated handling, selective deferral to human experts, and collaborative exploration. Under this approach, routine cases may be handled automatically, whereas uncertain or complex cases can be transferred to analysts with supporting information. The framework further argues that an AI system should recognize limitations in its own capabilities and escalate cases when its confidence is low or when the associated risk is high (Chhetri et al., 2024). Importantly, A2C remains a conceptual framework, with prototype development and empirical evaluation identified by the authors as future work.

This controlled-autonomy perspective is consistent with broader research on AI-augmented SOC. Srinivas et al. (2025) emphasize the importance of balancing automation with human oversight, particularly because AI systems may face limitations involving explainability, generalization, adversarial robustness, hallucinations, and other unreliable outputs. Accordingly, AI augmentation should not be interpreted simply as maximizing autonomous execution. Instead, the level of automation should reflect factors such as task complexity, system confidence, and the operational consequences of an incorrect decision, while preserving human involvement where expert judgment is required.

For the proposed AI-Based SOAR tool, this creates an important architectural principle: **intelligent analysis and automated execution should be connected through a controlled decision layer rather than directly coupled without supervision**. AI/ML can assist with alert analysis, contextual risk assessment, prioritization, and response recommendations, while SOAR provides the mechanisms for executing approved operational actions. The specific machine-learning approaches proposed for alert triage and prioritization are examined in the following section.

2.4 Review of Existing ML Triage Systems

The application of machine learning to SOC alert triage has evolved from supervised classification of individual security events toward context-aware, feedback-driven, and adaptive prioritization mechanisms. Existing approaches differ in the information they use, the form of prioritization they produce, the extent of analyst involvement, and their assumptions about operational resources. This section reviews representative systems that illustrate this progression and evaluates their relevance to intelligent alert handling.

2.4.1 Supervised Event Classification and Actionability Assessment

An early approach to automating SOC event handling was proposed by Gupta et al. (2019), who developed a deep-learning-based classification method using event data obtained from a real Security Operations Center. The dataset contained 39,427 labeled security events, where the labels indicated whether SOC analysts had notified the customer about an event. These labels can therefore be interpreted as an operational proxy for actionability rather than as direct malicious/benign ground truth. The authors performed feature engineering and selection before training a deep neural network to distinguish notified from non-notified events (Gupta et al., 2019).

The study demonstrated the feasibility of learning patterns associated with historical analyst decisions from operational SOC data. However, its results also illustrate the challenges associated with imbalanced alert populations. For the notified class, the proposed model achieved a recall of 0.92 but a precision of only 0.39, indicating that it identified most historically notified events while still producing a considerable proportion of false-positive predictions for that class. The post-tuned model reported an AUC of 92.67% on the labeled dataset (Gupta et al., 2019). Despite the use of the term *prioritization* in the study, the implemented output is primarily a binary notified/non-notified classification rather than a continuously ranked or dynamically updated alert queue.

A more operationally oriented approach was presented in a preprint study by Gelman et al. (2023) through the **That Escalated Quickly (TEQ)** framework. TEQ generates both alert-level and incident-level actionability scores and uses these scores for incident prioritization, false-positive suppression, and within-incident alert ordering. In a deployment simulation based on real data from a beta Managed Detection and

Response product, reordering incidents according to TEQ scores reduced the average queue time of actionable incidents by approximately 22.9% in the shortest evaluated queue configuration. Furthermore, a high-recall decision threshold suppressed 54% of false-positive incidents while capturing 95.1% of actionable incidents (Gelman et al., 2023). These results suggest that ML-based scoring can influence operational workload in addition to classification performance. Nevertheless, the reported improvements were obtained through a simulated deployment rather than a controlled live SOC deployment, and TEQ does not extend the prioritization decision into automated SOAR response execution.

2.4.2 Context-Aware Alert Triage and Prioritization

A limitation of approaches based predominantly on individual event attributes is that similar alerts may have different meanings depending on the surrounding activity. Liu et al. (2022) addressed this issue through **Context2Vector**, a context-representation-learning framework designed to support security-event triage. The system constructs behavioral contexts from event sequences and applies representation learning and deviation detection to generate interpretable contextual features that can support event risk ranking.

Context2Vector was evaluated using an enterprise-scale dataset collected from heterogeneous security sources, including IDS/IPS, EDR, sandbox, mail gateway, WAF, and UEBA systems. Across the prewar, warfare, and peacetime stages, the dataset contained approximately 2.45 million events (Liu et al., 2022). The study illustrates why an alert's isolated severity or signature may be insufficient for prioritization. For example, Remote Code Execution alerts occurring within an internal vulnerability-scanning context may reflect expected scanning activity, whereas similar vulnerability alerts occurring alongside authentication and login activity may indicate a potentially more concerning security context requiring further investigation. Thus, the operational significance of an alert can depend on its surrounding behavioral context rather than on the alert signature alone.

In its Top-N risk-ranking evaluation, Context2Vector improved attacker-IP recall by up to 2.25 times compared with traditional experience-driven approaches when evaluation was restricted to the top 500 events (Liu et al., 2022). This result should be interpreted specifically as an improvement in identifying attacker-associated sources within a limited investigation set, rather than as a direct 2.25-fold reduction in false-positive alerts. Although Context2Vector demonstrates the value of contextual representation for event classification and risk ranking, the evaluated framework focuses on triage and does not extend the resulting risk information into automated SOAR response execution.

Wang et al. (2024) extended context-aware prioritization by incorporating analyst feedback into **AlertPro**. The framework combines alert properties with contextual and historical features, initially ranks alerts using Isolation Forest, and subsequently re-ranks them through an active-learning process informed by analyst validation. Reinforcement-learning algorithms, including D3QN, A2C, and PPO2, were evaluated to support this feedback-driven re-ranking process (Wang et al., 2024).

AlertPro was evaluated across five datasets containing multi-step attacks. Its feedback mechanism allows analyst validation to influence subsequent alert rankings rather than treating the initial anomaly score as final. Across the evaluated datasets, AlertPro-A2C achieved average cumulative precision and cumulative recall values of 96.32% and 98.47%, respectively, compared with 64.65% and 66.03% for the Isolation Forest baseline (Wang et al., 2024). These results indicate that combining contextual information with iterative expert feedback can improve the concentration of genuine attack alerts within the set prioritized for analyst investigation. However, the evaluated framework remains focused on alert analysis and prioritization; response generation and SOAR-based incident-response execution are outside its evaluated scope.

2.4.3 Adaptive and Resource-Aware Prioritization

Alert prioritization can also be formulated as a resource-allocation problem because SOC analysts may not have sufficient capacity to investigate every alert generated by security systems. Chavali et al. (2024) addressed this problem using off-policy actor-critic deep reinforcement learning (DRL). Their proposed TD3-AP and SAC-AP methods model interactions between an adversary generating attack-related alerts and a defender operating under a limited alert-investigation budget. A game-theoretic double-oracle procedure is then used to derive adaptive alert-prioritization strategies.

The proposed methods were evaluated using MQTT-IoT-IDS2020, DARPA 2000 LLDOS 1.0, and CSE-CICIDS2018, with Snort used to generate IDS alerts. The primary evaluation measure is the defender's loss, which reflects the effectiveness of the defender's prioritization strategy under constrained investigation resources. Under one evaluated MQTT-IoT-IDS2020 configuration with an attack budget of 125, TD3-AP reduced the defender's loss by 50% relative to DDPG-MIX, while the magnitude of improvement varied under other resource configurations (Chavali et al., 2024). The reported testing time for TD3-AP was approximately 300 microseconds across the evaluated datasets; however, this value represents model testing or inference time rather than end-to-end incident-response latency.

The resource-aware formulation is significant because it moves beyond assigning isolated severity values and considers how limited investigation capacity should be allocated among competing alerts. Nevertheless, the evaluation is based on three publicly available datasets and IDS-generated alerts, and the authors identify broader real-world experimentation as an area for future work (Chavali et al., 2024). In addition, the framework focuses on prioritization and does not connect its learned allocation policy to automated SOAR response execution.

2.4.4 Comparative Analysis of Existing Approaches

The reviewed studies illustrate an evolution in ML-supported SOC triage from supervised classification toward context-aware, feedback-driven, and resource-aware prioritization. Gupta et al. (2019)

use supervised learning to identify patterns associated with historical analyst notification decisions, while Gelman et al. (2023) extend ML scoring toward incident queue management and false-positive suppression. Liu et al. (2022) incorporate behavioral context into event risk assessment through Context2Vector, whereas Wang et al. (2024) combine contextual information with iterative analyst feedback to support dynamic alert re-ranking. Chavali et al. (2024) further formulate prioritization as an adaptive resource-allocation problem under constrained investigation capacity.

Table 2-1 summarizes the main differences among these approaches in terms of their data environments, AI/ML methods, contextual or feedback mechanisms, prioritization outputs, response integration, and principal limitations.

Table 2-1. Comparative Analysis of Existing ML-Based SOC Triage Approaches

Study	Data / Environment	AI/ML Approach	Context / Feedback	Prioritization Output	SOAR / Response	Main Limitation
Gupta et al. (2019)	Real SOC-labeled SOD events	DNN supervised classification	Engineered event features; no online feedback	Notified / non-notified classification	No	Binary output; low minority-class precision
Gelman et al. (2023)	Real MDR data; deployment simulation	RF + NN actionability scoring	Content/context features; historical labels	Incident and alert scores, suppression	No	Preprint; operational effects simulated
Liu et al. (2022)	Enterprise-scale heterogeneous event data	Context representation + deviation detection	Behavioral context; expert annotation	Contextual risk ranking	No	No downstream response execution
Wang et al. (2024)	Five multi-step attack datasets	Isolation Forest + RL-based active learning	Context/history + iterative analyst feedback	Dynamic alert re-ranking	No	Stops at triage/investigation
Chavali et al. (2024)	Three public IDS datasets	TD3-AP / SAC-AP DRL	Resource/state information; no analyst feedback	Resource-aware investigation priority	No	No real-world SOC validation or SOAR execution

Overall, the reviewed approaches address different aspects of intelligent alert handling, including actionability assessment, contextual risk analysis, false-positive suppression, analyst-guided re-ranking, and investigation-resource allocation (Chavali et al., 2024; Gelman et al., 2023; Gupta et al., 2019; Liu et al., 2022; Wang et al., 2024). Their differences in analytical scope, operational output, and level of downstream integration provide the basis for the broader problem synthesis presented in the following section.

2.5 Gap Analysis and Problem Synthesis

The reviewed literature demonstrates substantial progress in applying AI/ML to different stages of SOC alert handling. Context2Vector shows that behavioral context can provide additional information for distinguishing and prioritizing security events beyond isolated alert attributes (Liu et al., 2022). AlertPro further demonstrates that contextual and historical features can be combined with analyst feedback to support continuous alert re-ranking (Wang et al., 2024). Chavali et al. (2024) extend this direction by formulating prioritization as an adaptive resource-allocation problem under limited investigation capacity. Collectively, these studies provide evidence for increasingly intelligent triage and prioritization mechanisms; however, their evaluated workflows primarily focus on event ranking, alert selection, or analyst investigation rather than downstream SOAR-controlled response execution.

Research on automated incident response addresses a complementary portion of the incident-handling pipeline. Karlzén and Sommestad (2023) review automatic and semi-automatic incident-response approaches and compare academic solutions with commercially available SOAR playbooks, highlighting research concerned with the inputs used for response decisions and the defensive actions that can be executed. Sworna et al. (2023) address orchestration more specifically through IRP2API, which maps predefined incident-response tasks to appropriate APIs across heterogeneous security tools. This approach supports tool and API selection once a response task has been defined, but it does not determine the required response directly from an alert's contextual risk, and automated execution of the mapped APIs is identified as future work. In contrast, Ismail et al. (2025) propose an agentic AI-based SOAR architecture capable of generating technical response steps, allowing human intervention, and executing validated commands through external tools. Their pilot implementation primarily evaluates automation and orchestration after enriched security alerts have been received, rather than the upstream problem of learning contextual alert risk and dynamically prioritizing incoming alerts.

Human oversight introduces an additional consideration when intelligent analysis is connected to response automation. Chhetri et al. (2024) propose flexible autonomy in which routine cases may be automated, uncertain cases may be selectively deferred to analysts, and novel situations may require collaborative human-AI exploration. This provides a conceptual basis for adjusting the level of automation according to factors such as confidence, complexity, and risk. However, the A2C framework remains conceptual, with prototype implementation and empirical evaluation identified as future work. Therefore,

although the reviewed literature provides mechanisms for intelligent triage, adaptive prioritization, human-AI decision support, and automated response, these capabilities remain only partially integrated across the experimentally evaluated workflows reviewed in this chapter.

Broad AI-enabled SOC architectures have also been proposed to integrate data acquisition, intelligent security analysis, and operational response functions (Khayat et al., 2025). Consequently, the research gap should not be interpreted as an absence of AI-enabled SOC architectures, dynamic prioritization, human-AI collaboration, or automated response. Rather, the reviewed evidence indicates that **end-to-end empirical integration remains limited, particularly in connecting context-enriched alert risk and dynamic prioritization with controlled SOAR response-policy execution.**

Accordingly, the proposed project investigates how these complementary capabilities can be integrated into a coherent AI-Based SOAR tool. The proposed direction connects **context-enriched alert analysis, contextual risk scoring, dynamic prioritization, controlled response decision making, and SOAR-based execution** within a unified workflow. In this design, alert risk is intended not only to influence queue position but also to contribute to a response policy that determines whether an alert is suppressed, escalated for analyst investigation, submitted for human approval, or associated with an automated response. This integration provides the conceptual foundation for the system architecture presented in Chapter 3.

Chapter 3:

Proposed System Architecture

3.1 Introduction

This chapter presents the proposed **AI-Based Security Orchestration, Automation, and Response (SOAR) tool** and describes the architecture through which security alerts are collected, enriched, analyzed, prioritized, and connected to controlled response actions. The proposed tool combines conventional SOAR capabilities with AI/ML-based analysis to support context-aware alert assessment, dynamic prioritization, and policy-controlled incident response.

The architecture follows a modular design in which each component performs a distinct operational function. Security alerts are first collected and normalized through the ingestion layer, after which contextual information is added to improve alert interpretation. The enriched alerts are then processed by the machine-learning components to produce contextual risk information that supports prioritization, suppression, and response-policy decisions. High-impact response actions remain subject to predefined policy controls and human approval where necessary.

In addition to the functional modules, the proposed tool incorporates an **agent-oriented architecture** consisting primarily of an **Attacker Agent** and a **Defender Agent**, informed by the AI-driven Multi-Agent System approach presented by Agrawal et al. (2026). The Attacker Agent supports adaptive attack simulation within the controlled testing environment, whereas the Defender Agent coordinates intelligent defensive analysis and decision support. Detailed implementation, training, and evaluation of the agents are deferred to Chapter 4.

The following sections describe the high-level architecture, agent layer, ingestion and enrichment components, machine-learning triage and scoring engine, prioritization logic, automated response policy, and analyst feedback mechanism.

3.2 High-Level Architecture Overview

The proposed AI-Based Security Orchestration, Automation, and Response (SOAR) tool is designed as a modular pipeline that connects security-alert collection, contextual analysis, machine-learning-based risk assessment, dynamic prioritization, and controlled incident-response execution. The architecture builds on the complementary roles of SOAR and AI/ML identified in the literature: SOAR provides workflow orchestration, integration, and execution capabilities, while AI/ML can support data-driven analysis, classification, prioritization, and decision support (Khayat et al., 2025; Kinyua & Awuah, 2021). Rather than replacing conventional SOAR functions, the proposed tool introduces an intelligent decision layer between alert collection and response execution.

In addition to its functional organization, the proposed AI-Based SOAR tool adopts an **agent-oriented architecture** inspired by recent AI-driven multi-agent cybersecurity research. Two principal agent roles are considered: an **Attacker Agent**, which operates within the controlled testing environment to generate adaptive attack activity, and a **Defender Agent**, which supports intelligent alert analysis, contextual risk assessment, prioritization, and response decision making. The agent layer does not replace the functional modules of the architecture; rather, it provides an intelligent coordination layer through which selected analytical and decision-making functions can be performed. The detailed implementation, training procedures, learning algorithms, and interaction mechanisms of these agents are presented in Chapter 4 (Agrawal et al., 2026).

At a high level, the architecture begins with **security data sources**, which may include SIEM platforms, intrusion-detection systems, endpoint-security technologies, network-security controls, and other monitoring tools. Alerts generated by these sources are received through the **Ingestion and Connectors Module**, which provides a common entry point for heterogeneous security data. The module is responsible

for collecting and normalizing incoming alert information into a consistent representation that can be processed by the subsequent components of the tool.

The normalized alert is then passed to the **Context Enrichment Layer**. This layer supplements the original alert with additional information relevant to its security significance. Depending on the available environment and data sources, enrichment may include threat-intelligence indicators, asset criticality, user or identity information, source reputation, and relationships with recent or historical security events. The enriched alert is subsequently provided to the **Machine Learning Triage and Scoring Engine**, which analyzes the available features and produces a contextual risk assessment. The principal output of this stage is a risk score representing the estimated importance of the alert, together with any additional model output required by the prioritization process.

The resulting score is processed by the **Prioritization and Suppression Logic**, which determines how incoming alerts should be ordered and whether selected low-risk or redundant alerts are eligible for suppression or deprioritization according to predefined policy conditions. This stage converts model output into an operational priority that can be used to allocate analyst attention more effectively. High-risk alerts are therefore intended to receive greater operational attention than alerts assessed as low risk, while suppression decisions remain governed by configurable rules rather than by the machine-learning model alone.

After prioritization, the alert is passed to the **Automated Response Policy**. This component determines the appropriate level of response based on factors such as contextual risk, alert priority, confidence, and the potential operational impact of the available action. The proposed architecture separates intelligent analysis from response execution so that high-impact actions are not triggered directly by an unverified model prediction. Depending on the policy decision, an alert may be suppressed, monitored, escalated for analyst investigation, submitted for human approval, or forwarded to a SOAR workflow for automated execution. This controlled approach is consistent with human-AI teaming research that emphasizes flexible autonomy and human involvement in uncertain or high-risk situations (Chhetri et al., 2024).

Finally, the **Analyst Feedback Loop** records analyst decisions and investigation outcomes. Feedback may include confirmation or correction of alert priority, validation of the risk assessment, approval or rejection of recommended actions, and the final incident disposition. This information can be retained for subsequent analysis, threshold adjustment, and future model refinement rather than assuming unrestricted online learning during live operation.

The relationship between the agents and the functional pipeline can be summarized as follows:

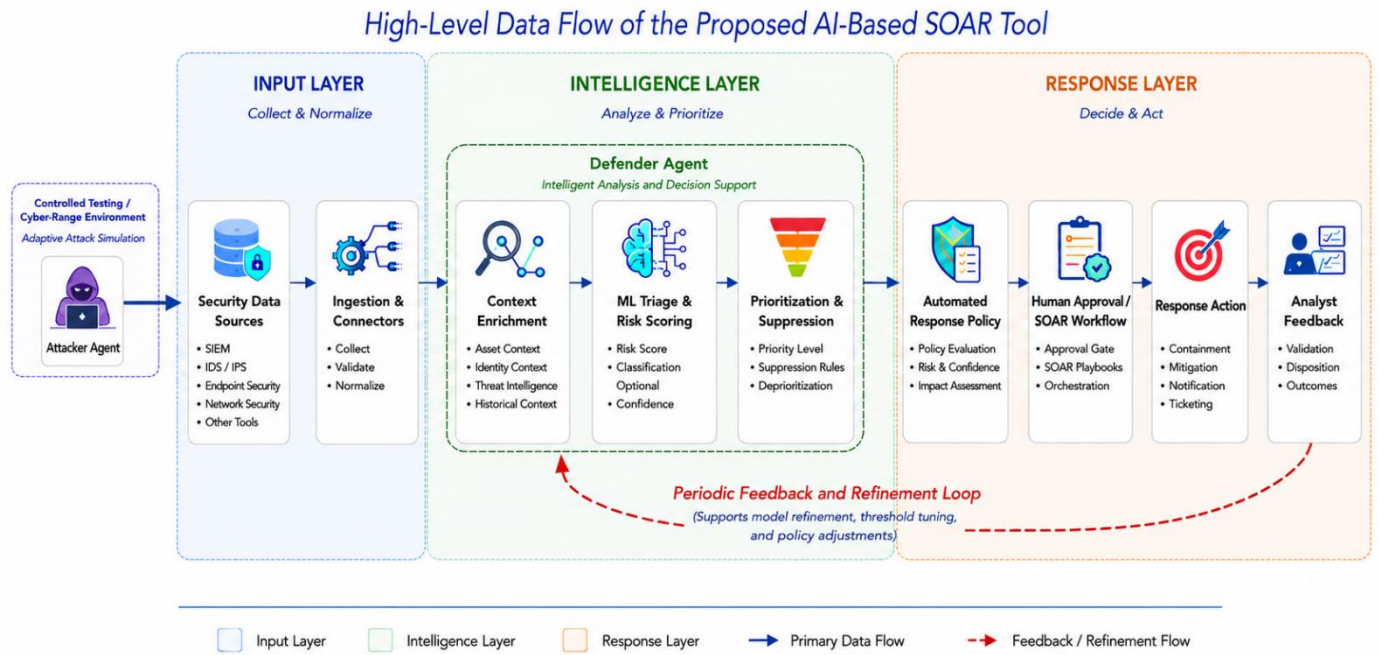


Figure 3-1. High-Level Architecture of the Proposed AI-Based SOAR Tool

3.2.1 Agent Layer Overview

The proposed AI-Based SOAR tool incorporates an **agent layer** to support adaptive attack simulation and intelligent defensive decision making. This design is informed by the AI-driven Multi-Agent System (MAS) presented by Agrawal et al. (2026), which employs attacker and defender agents within a controlled cyber-range environment. In the reference system, reinforcement-learning techniques are used to support adaptive attacker behavior, while machine-learning-based defender agents perform threat detection and support incident-response decisions.

Within the proposed tool, the **Attacker Agent** is primarily associated with the testing and evaluation environment rather than with the operational SOAR workflow itself. Its purpose is to generate adaptive or multi-stage attack activity that produces security events and alerts for evaluating the defensive components of the system. The Attacker Agent can interact with the controlled environment and modify its behavior according to environmental state and observed defensive responses. The detailed state representation, available actions, reward mechanism, and reinforcement-learning approach are deferred to Chapter 4.

The **Defender Agent** represents the principal intelligent defensive role within the proposed architecture. It receives security information produced by the ingestion and contextual-enrichment stages and interacts with the machine-learning components responsible for risk assessment, classification where applicable, and alert prioritization. The Defender Agent also uses the resulting analytical information to support response-policy evaluation. This role is consistent with Agrawal et al. (2026), where defender agents employ machine-learning techniques for threat detection and participate in automated incident-response decision making.

The agent layer is therefore integrated with, rather than separated from, the functional architecture. The Attacker Agent provides adaptive security activity within the controlled testing environment, while the Defender Agent coordinates intelligent defensive analysis and decision support before operational actions are executed through SOAR workflows and applicable human-approval controls.

Chapter 3 defines the architectural responsibilities and placement of these agents, while Chapter 4 describes their implementation, machine-learning and reinforcement-learning techniques, training procedures, interaction mechanisms, and experimental evaluation.

The following sections describe each architectural component and its role within the proposed workflow in greater detail.

3.3 Ingestion and Connectors Module

The **Ingestion and Connectors Module** serves as the entry point of the proposed AI-Based SOAR tool. Its primary purpose is to receive security alerts and associated telemetry from heterogeneous security technologies and transform them into a consistent representation that can be processed by the downstream analytical components. Modern SOC environments commonly combine multiple products with different data formats, interfaces, and representations, which can complicate unified security analysis and orchestration (Kinyua & Awuah, 2021). Accordingly, the proposed module is designed to separate source-specific integration concerns from the internal processing logic of the tool.

Security information may originate from different categories of technologies, including SIEM platforms, intrusion-detection systems, endpoint-security solutions, network-security controls, and other monitoring or detection mechanisms. Depending on the implementation environment, these sources may expose alerts through REST APIs, webhooks, message streams, log files, or other supported interfaces. Rather than requiring the remaining components of the architecture to understand each vendor-specific format, individual connectors are responsible for communicating with the corresponding source and translating incoming data into a common internal alert structure.

The normalization process preserves the security-relevant information contained in the original event while standardizing fields required by the proposed tool. A normalized alert may contain attributes such as an alert identifier, timestamp, source system, alert type, original severity, source and destination information, affected user or asset, event description, and any available indicators of compromise. The module should also retain a reference to the original alert or raw event data when available so that normalized information can be traced back to its source during investigation or validation.

Before forwarding an alert to subsequent components, the module performs basic validation and preprocessing. This may include verifying required fields, converting timestamps into a consistent format, standardizing field names and categorical values, removing malformed or incomplete records where appropriate, and identifying duplicate events according to predefined criteria. Such operations are intended to improve data consistency rather than to determine whether an alert is malicious or important. Risk assessment and prioritization remain the responsibility of later modules.

The use of connector-based integration also supports interoperability with multi-vendor security environments. SOAR platforms depend on integrations with external technologies to query information and initiate operational actions, while differences among vendor interfaces can create substantial integration effort (Bridges et al., 2023; Sworna et al., 2023). In the proposed architecture, this problem is addressed through an abstraction layer in which each connector implements the communication requirements of a particular data source while exposing a standardized output to the rest of the tool. New sources can therefore be incorporated by developing or configuring an additional connector without redesigning the machine-learning or prioritization components.

Each normalized record should also maintain **source provenance**. Information such as the originating security product, connector identifier, ingestion timestamp, and original alert identifier can be retained to

support traceability and later response decisions. This is particularly important when automated or analyst-approved actions must be related back to the evidence that triggered the workflow.

The output of the Ingestion and Connectors Module is therefore a validated and normalized alert object that provides a common foundation for further processing. The normalized alert is forwarded to the **Context Enrichment Layer**, where additional information from internal and external sources is attached before machine-learning-based risk assessment is performed.

A simplified representation of the module is as follows:

Simplified Pipeline Representation of Data Parsing, Normalization, and Enrichment

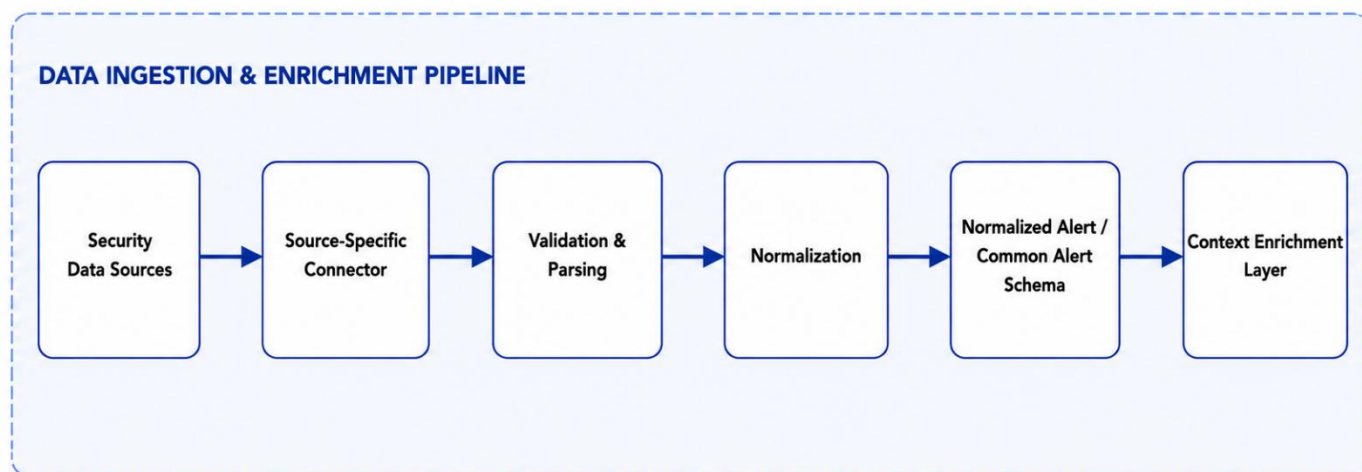


Figure 3-2: Ingestion and Context Enrichment Sub-Module Architecture

3.4 Context Enrichment Layer

The **Context Enrichment Layer** is responsible for extending the normalized alert with additional information that can improve its interpretation before machine-learning-based risk assessment is performed. An alert considered in isolation may provide only limited information about its operational significance, whereas surrounding behavioral, organizational, and threat-intelligence context can help distinguish routine activity from events that warrant greater attention. Context-aware analysis has been shown to improve security-event triage by incorporating relationships and surrounding activity rather than relying solely on individual alert attributes (Liu et al., 2022).

The proposed layer receives the normalized alert produced by the Ingestion and Connectors Module and enriches it using information available from internal and external security sources. The enrichment process is modular so that the tool can operate with different combinations of contextual information depending on the deployment environment. Potential enrichment categories include **asset context, identity context, threat-intelligence context, historical context, and behavioral or temporal context**.

Asset context provides information about the system or resource associated with an alert. Relevant attributes may include asset type, business criticality, network location, operating environment, or known exposure information. For example, an alert affecting a critical production server may require different operational treatment from a similar alert affecting a low-criticality test system. Asset information therefore provides organizational context that cannot necessarily be inferred from the original security alert alone.

Identity context may include information related to the user, account, or service associated with an event. Depending on data availability, this may include account type, privilege level, authentication history, or whether the identity is associated with a sensitive resource. Such information can help distinguish events involving ordinary accounts from those involving privileged or otherwise high-impact identities.

Threat-intelligence context enriches observable indicators contained in an alert, such as IP addresses, domains, URLs, or file hashes, with information obtained from available reputation or threat-intelligence sources. SOAR environments commonly integrate threat-intelligence services to support investigation and contextualization of incoming alerts (Bridges et al., 2023). In the proposed tool, this information may include reputation status, indicator type, known malicious associations, or other metadata returned by the available intelligence source.

The layer may also incorporate **historical and behavioral context**. Historical information can indicate whether similar alerts, indicators, users, or assets have appeared previously, while temporal relationships can identify activity occurring within a relevant time window. This is important because the significance of an event can depend on surrounding behavior. Context2Vector, for example, demonstrates that similar security events can have different operational interpretations depending on the behavioral context in which they occur (Liu et al., 2022). The proposed architecture therefore allows recent event history and related activity to contribute contextual information before scoring.

The enrichment process does not itself assign the final alert priority or determine the response action. Its purpose is to create an **enriched alert representation** containing both the original normalized attributes and the contextual information required by subsequent analysis. Each added attribute should retain its source and availability status where practical so that the system can distinguish between verified contextual information, missing values, and unavailable enrichment sources.

Because external or internal context may occasionally be unavailable, the architecture should not assume that every alert will contain every enrichment field. Missing contextual information can be represented explicitly and handled during later feature preparation and model development. This design allows the tool to degrade gracefully when a particular enrichment service or organizational data source is unavailable instead of preventing the alert from proceeding through the pipeline.

The output of the Context Enrichment Layer is an enriched alert object that is forwarded to the **Machine Learning Triage and Scoring Engine**. The subsequent engine transforms the available alert and contextual attributes into the features required for machine-learning-based analysis and produces the risk information used by the prioritization component.

A simplified representation of the enrichment process is:

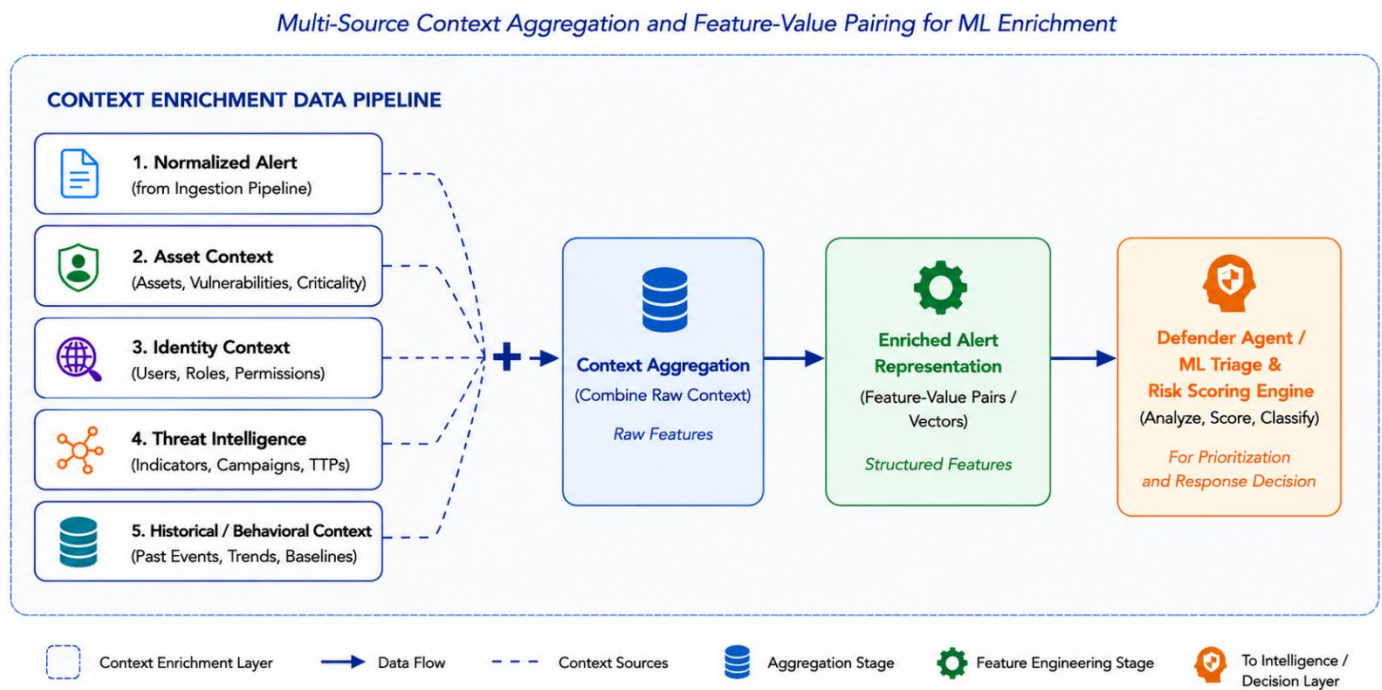


Figure 3-3: Context Enrichment Data Parsing Pipeline

3.5 Machine Learning Triage and Scoring Engine

The **Machine Learning Triage and Scoring Engine** is the analytical core of the proposed AI-Based SOAR tool. Its purpose is to evaluate each enriched alert using both its original security attributes and the contextual information supplied by the Context Enrichment Layer. Rather than relying exclusively on static severity values or manually defined thresholds, the engine applies a trained machine-learning model to estimate the operational risk associated with an alert and provide structured information to the subsequent prioritization component.

From the agent-oriented perspective, the Machine Learning Triage and Scoring Engine represents a principal analytical capability of the **Defender Agent**. The agent receives the enriched alert representation, invokes the selected machine-learning models, and uses their outputs to support contextual risk assessment and subsequent prioritization. The Defender Agent therefore coordinates the analytical process, while the ML engine remains responsible for producing the model-derived risk, classification, and confidence-related outputs defined by the architecture.

The engine receives an enriched alert and transforms the available attributes into a **model-ready feature representation**. Depending on the selected model and dataset, these features may include properties derived from the original alert, asset and identity characteristics, threat-intelligence indicators, historical observations, and behavioral or temporal context. Context-aware security research has shown that incorporating surrounding activity can improve the interpretation and prioritization of alerts compared with evaluating isolated event attributes alone (Liu et al., 2022). Similarly, feedback-driven approaches such as AlertPro demonstrate the value of combining alert properties with contextual and historical features for alert ranking (Wang et al., 2024).

The primary output of the engine is a **contextual risk score** representing the estimated operational significance of the alert. For architectural purposes, the score can be normalized to a consistent range, such as 0 to 1, where higher values indicate greater estimated risk. The exact mathematical interpretation of this score depends on the machine-learning technique selected during model development. For example, it may

represent a predicted probability, anomaly-related score, or another model-derived measure that is subsequently normalized for operational use. The detailed model-selection, training, feature-engineering, and score-calibration procedures are described in Chapter 4.

In addition to the primary risk score, the engine may produce a **predicted class or actionability label** when supported by the selected model. Such a label may distinguish, for example, between alerts estimated to require further investigation and those considered lower priority. Previous SOC research has demonstrated the feasibility of learning patterns associated with historical analyst handling decisions through supervised classification (Gupta et al., 2019). However, the proposed architecture does not rely on a binary classification alone; the continuous risk score remains the principal input to dynamic prioritization because it provides a finer basis for ordering alerts with different estimated levels of importance.

A **confidence indicator** may also be retained when it can be derived reliably from the selected model. This value is conceptually distinct from the risk score. Risk represents the estimated security significance of an alert, whereas confidence reflects the certainty associated with the model's assessment. Maintaining this distinction is particularly important for controlled automation. A high-risk prediction accompanied by low confidence may require analyst validation, while a high-confidence assessment may support a different response-policy decision. The exact method used to estimate or calibrate confidence is deferred to the model-development stage and will depend on the final algorithm.

Where supported by the selected technique, the engine may additionally expose explanatory information such as the features that contributed most strongly to the resulting score. Such information can support analyst interpretation and later validation, but explainability is treated as a supporting output rather than as a substitute for the risk assessment itself.

The engine is deliberately separated from both prioritization and response execution. It does not directly suppress alerts, reorder the analyst queue, or initiate containment actions. Instead, it produces analytical outputs that are consumed by later architectural components. This separation prevents a raw machine-learning prediction from being translated directly into a potentially high-impact response without additional policy checks.

The principal outputs of the Machine Learning Triage and Scoring Engine are therefore:

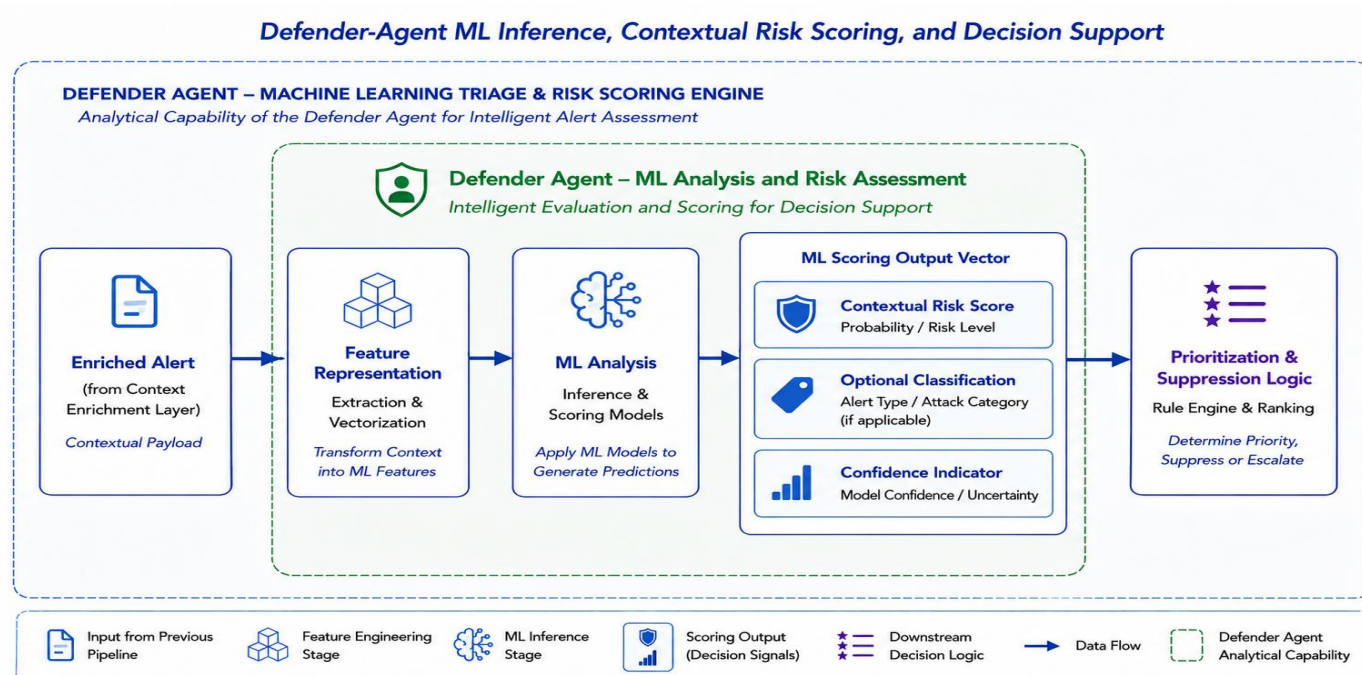


Figure 3-4: Machine Learning Triage and Risk Scoring Architecture

The **contextual risk score** is the mandatory analytical output of the proposed design, while classification and confidence are supporting outputs whose implementation depends on the final machine-learning model. This design keeps the architecture sufficiently flexible to accommodate different algorithms while preserving a consistent interface with the subsequent prioritization component.

The **Prioritization and Suppression Logic** converts the analytical outputs of the Machine Learning Triage and Scoring Engine into operational decisions about alert ordering and analyst attention. While the ML engine estimates the contextual risk associated with each alert, this module determines how that score should influence the active investigation queue and whether selected alerts are eligible for suppression or deprioritization. Separating these responsibilities allows model predictions to be interpreted through explicit operational policies rather than being applied directly to incident handling.

Within the agent layer, the **Defender Agent** uses the outputs of the ML Triage and Scoring Engine as inputs to the prioritization process. The agent does not independently discard alerts; suppression and deprioritization remain governed by the explicit policy conditions and safeguards defined by this module.

The principal input to this module is the **contextual risk score** produced by the ML engine. Supporting information, such as the model confidence indicator, original alert severity, asset criticality, or other policy-relevant attributes, may also be considered where available. The resulting operational priority can be represented through ordered categories such as **Low, Medium, High, and Critical**, or through a continuously ranked queue. The exact thresholds separating these categories are configurable and should be established during system evaluation rather than assumed to be universally applicable across SOC environments.

Dynamic prioritization is intended to ensure that an alert's position is determined by its current contextual assessment rather than solely by the static severity assigned by the originating security product. As new alerts enter the system or additional context becomes available, their relative positions can be recalculated according to the applicable prioritization policy. This design is consistent with prior research demonstrating that contextual information and analyst feedback can improve alert ranking beyond an initial anomaly or severity score (Wang et al., 2024).

The module also includes **suppression and deprioritization logic** for alerts that satisfy predefined low-risk conditions. Suppression is intended to reduce unnecessary analyst workload by preventing selected alerts from repeatedly occupying investigation capacity. However, the proposed architecture does not allow the machine-learning score alone to permanently discard an alert. A suppression decision should depend on explicit policy conditions, which may include a sufficiently low contextual risk score, adequate model confidence, duplicate or repeated-event status, absence of high-criticality assets or identities, and the lack of other escalation conditions. Machine-learning-based alert scoring has previously been explored for false-positive suppression and queue management, illustrating the potential operational value of combining scoring with controlled filtering mechanisms (Gelman et al., 2023).

To reduce the risk of hiding significant incidents, the module distinguishes between **suppression** and **deprioritization**. Deprioritized alerts remain available for later inspection but are placed below higher-risk events in the investigation queue. Suppressed alerts are excluded from the primary queue according to the active policy but should remain recorded for audit, retrospective analysis, and possible reconsideration if related evidence emerges. This distinction preserves traceability while reducing immediate analyst workload.

The prioritization logic also supports **policy overrides**. Certain alerts may be prevented from suppression regardless of their model score when they involve critical assets, privileged identities, known high-risk indicators, or other organization-defined conditions. Similarly, low-confidence predictions can be

routed for analyst review instead of being automatically suppressed. These safeguards ensure that operational rules remain capable of constraining the influence of the ML model when the potential consequence of an incorrect prioritization decision is high.

The main output of this module is therefore an **operationally prioritized alert** associated with a priority level and a handling status. The handling status may indicate that the alert should remain in the investigation queue, be deprioritized, be suppressed according to policy, or be forwarded for additional response evaluation. The module does not itself execute containment or remediation actions; alerts requiring further action are passed to the **Automated Response Policy** described in the next section.

A simplified representation of the process is:

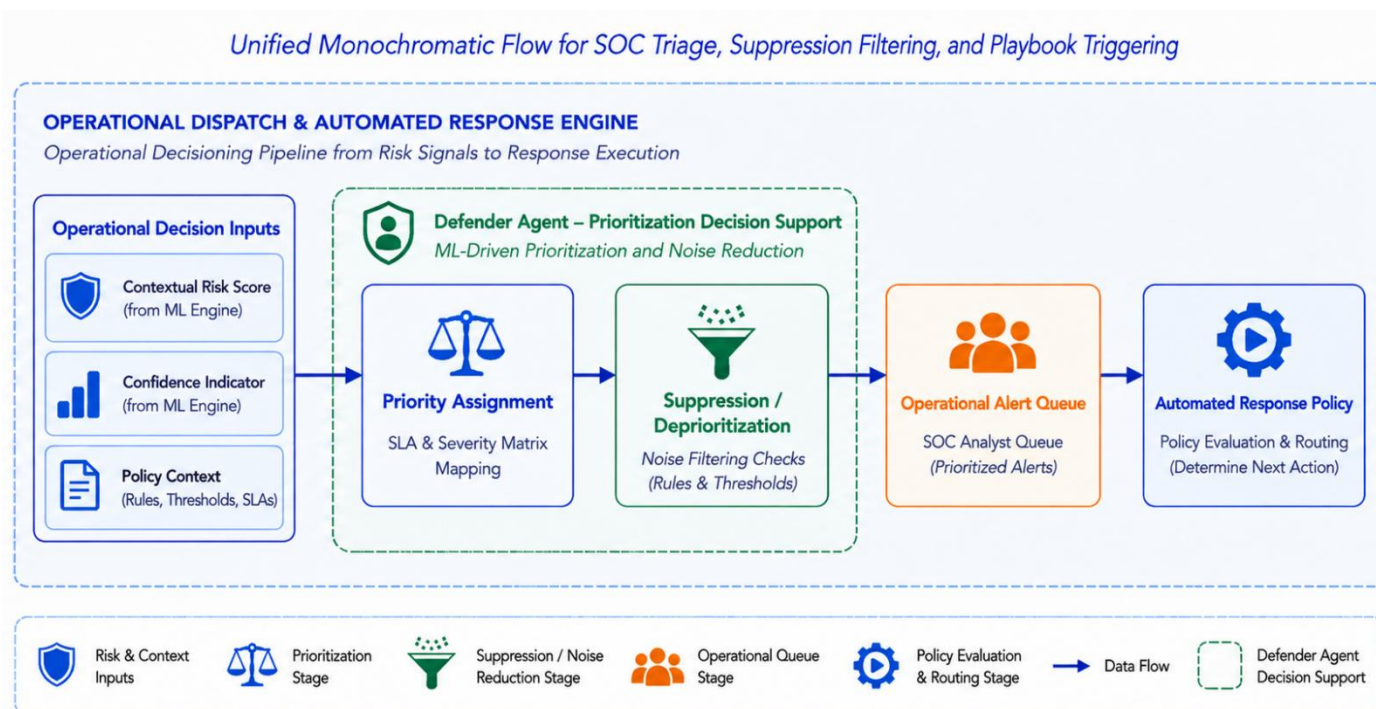


Figure 3-5: Alert Prioritization, Routing, and Automated Response Pipeline

3.7 Automated Response Policy

The **Automated Response Policy** determines how a prioritized alert should be handled after its contextual risk and operational priority have been established. Its purpose is to translate analytical and prioritization outputs into a controlled response decision while preventing high-impact actions from being triggered directly by a machine-learning prediction. This module therefore acts as a policy boundary between intelligent alert assessment and SOAR-based execution.

From the agent perspective, the **Defender Agent** uses the outputs of the analytical and prioritization components to support response-policy evaluation. However, the agent does not bypass the control mechanisms of the SOAR architecture. High-impact actions remain subject to predefined policy constraints and, where required, explicit human approval before SOAR-based execution.

The policy receives information from the preceding modules, including the **contextual risk score**, assigned priority level, model confidence where available, alert handling status, and relevant contextual attributes such as asset criticality, identity sensitivity, threat-intelligence findings, and the type of response action under consideration. These inputs are evaluated against predefined response rules to determine the appropriate level of automation.

The proposed tool distinguishes among several response outcomes. A low-risk alert may require no immediate response beyond continued monitoring or recording. Alerts requiring investigation may be escalated to an analyst without triggering automated containment. Where a response action is appropriate but may have significant operational consequences, the policy can require **human approval** before the corresponding SOAR workflow is executed. Lower-risk and well-defined actions may be eligible for automated execution when the applicable policy conditions are satisfied.

The decision to automate an action should therefore consider not only the estimated security risk of the alert but also the potential impact of the response itself. For example, performing an enrichment query or creating an incident ticket generally carries less operational risk than disabling a privileged account, isolating a production endpoint, or modifying a firewall policy. This distinction supports a **risk-aware level of autonomy**, in which the amount of human involvement depends on the uncertainty and consequence associated with the decision. Human-AI teaming research similarly emphasizes flexible autonomy and selective deferral when confidence is low or operational risk is high (Chhetri et al., 2024).

For architectural purposes, response handling can be represented through the following policy states:

- **Monitor / Record:** retain the alert and associated evidence without initiating a containment action.
- **Analyst Investigation:** forward the alert to the analyst queue for further validation.
- **Human Approval Required:** generate a recommended response but require explicit analyst authorization before execution.
- **Automated Response:** execute an approved low-risk or well-defined response workflow when the policy conditions are satisfied.

The exact thresholds and rules used to select among these states are configurable and should be determined during implementation and evaluation. A critical alert does not automatically imply unrestricted automated response. For example, a high-risk alert affecting a critical production asset may instead require mandatory human approval because the consequence of an incorrect containment action could be substantial. Similarly, low model confidence can override otherwise eligible automation and route the case to analyst review.

Once a response decision has been made, the module selects or invokes the appropriate **SOAR workflow or playbook**. The workflow may perform actions such as threat-intelligence enrichment, ticket creation, notification, blocking of a malicious indicator, account-related actions, endpoint containment, or other security operations supported by the connected technologies. SOAR playbooks provide a mechanism for coordinating defensive actions across multiple security controls, while APIs enable these workflows to interact with external security products (Karlzén & Sommestad, 2023; Sworna et al., 2023).

All policy decisions and response actions should be logged for traceability. The recorded information may include the alert identifier, risk score, priority, selected policy state, whether human approval was required, the executed workflow, action outcome, and relevant timestamps. This audit trail allows analysts to review why a response was selected and provides information that can later contribute to system evaluation and the Analyst Feedback Loop.

A simplified representation of the response process is:

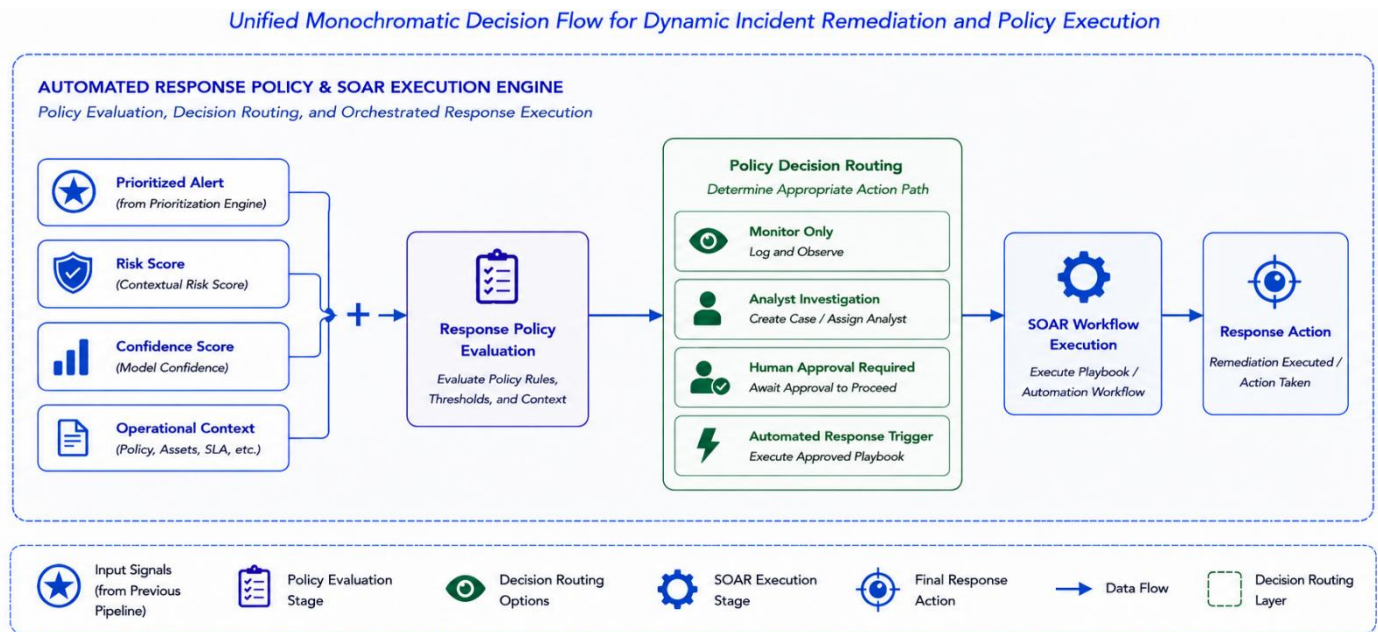


Figure 3-6: Automated Response Policy Evaluation and SOAR Workflow Pipeline

The output of this module is therefore a controlled response decision and, where appropriate, an executed SOAR workflow. The result of the action and any analyst intervention are subsequently recorded and forwarded to the **Analyst Feedback Loop**.

3.8 Analyst Feedback Loop

The **Analyst Feedback Loop** provides a mechanism for incorporating human validation and investigation outcomes into the proposed AI-Based SOAR tool. Its purpose is not to allow the system to modify its behavior autonomously during live operation, but to capture analyst judgments and response outcomes so that model performance, prioritization policies, and response rules can be evaluated and refined over time.

Feedback may be generated at several points in the workflow. During alert investigation, an analyst may confirm or reject the model's risk assessment, adjust the assigned priority, identify whether the alert represents a true or false positive, or provide a final incident disposition. During response handling, the analyst may approve or reject a recommended action, select an alternative response, or record whether an executed workflow achieved the intended result. These decisions provide operational evidence about the quality of both the analytical and response components of the system.

The proposed tool records feedback together with the alert information that produced the original decision. A feedback record may include the alert identifier, contextual risk score, assigned priority, model confidence where available, analyst-adjusted priority, final incident classification, selected response action, approval or rejection decision, response outcome, and relevant timestamps. Maintaining this linkage allows later analysis to compare machine-generated recommendations with human decisions and observed operational results.

Analyst feedback can support several improvement activities. First, it can be used to assess whether the machine-learning model consistently overestimates or underestimates risk for particular types of alerts. Second, accumulated analyst decisions can provide additional labeled examples for periodic model retraining or validation. Third, feedback can support adjustment of prioritization thresholds and suppression policies when operational results indicate that the current configuration produces excessive false positives, unnecessary escalations, or missed high-priority events. Feedback-driven alert prioritization has been

explored in previous research, where analyst validation is used to influence subsequent alert rankings (Wang et al., 2024).

The feedback mechanism also supports the controlled-autonomy principle adopted by the proposed architecture. Human-AI teaming research emphasizes that analysts and AI systems should be able to share information and adjust decision authority according to the situation, particularly when uncertainty or operational risk is high (Chhetri et al., 2024). In the proposed tool, analyst intervention therefore serves both an immediate operational role and a longer-term learning role by identifying cases in which automated assessment or policy behavior requires refinement.

The architecture distinguishes between **operational feedback** and **model updating**. Operational feedback is collected continuously as analysts interact with alerts and response recommendations. Model retraining, threshold modification, or policy adjustment, however, should occur through a controlled offline process after sufficient feedback has been reviewed and validated. This separation prevents unverified analyst inputs or isolated decisions from immediately altering the behavior of the production model.

The collected analyst feedback can also support the periodic offline evaluation and refinement of the decision models associated with the **Defender Agent**. This refinement remains a controlled process and does not imply unrestricted online modification of agent behavior during live operation.

All feedback should remain traceable to the corresponding alert and response workflow. This creates an auditable record that can support performance evaluation, error analysis, and future model development. Over time, the accumulated feedback can help identify recurring disagreement between the AI/ML engine and analysts, assess the suitability of automation policies, and guide revisions to the tool.

A simplified representation of the feedback process is:

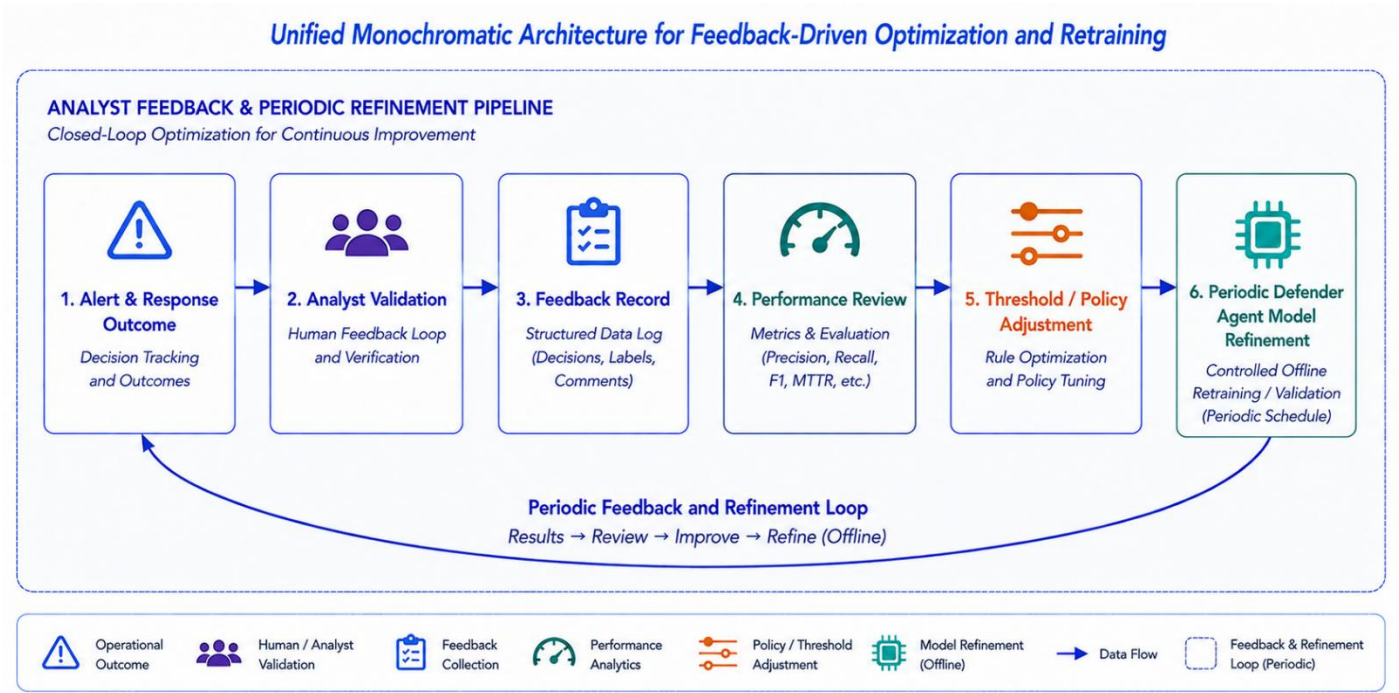


Figure 3-7: Analyst Feedback, Policy Adjustment, and Periodic Model Refinement Loop

The Analyst Feedback Loop therefore closes the proposed architectural cycle by connecting operational outcomes back to system evaluation and future improvement. This allows the tool to remain human-supervised while providing a structured basis for refining alert scoring, prioritization, and response decision making.

References

- Agrawal, A., Nadeem, M., Al Nuaim, A. and Al Nuaim, A. (2026) 'Artificial intelligence driven multi agent framework for adaptive cyber attack simulation and automated incident response in cyber range environments', *Scientific Reports*, article in press. doi:10.1038/s41598-026-45937-9.
- Bridges, R.A., Rice, A.E., Oesch, S., Nichols, J.A., Watson, C., Spakes, K., Norem, S., Huettel, M., Jewell, B., Weber, B., Gannon, C., Bizovi, O., Hollifield, S.C. and Erwin, S. (2023) 'Testing SOAR tools in use', *Computers & Security*, 129, 103201. doi:10.1016/j.cose.2023.103201.
- Chavali, L., Krishnan, A., Saxena, P., Mitra, B. and Chivukula, A.S. (2024) 'Off-policy actor-critic deep reinforcement learning methods for alert prioritization in intrusion detection systems', *Computers & Security*, 142, 103854. doi:10.1016/j.cose.2024.103854.
- Chhetri, M.B., Tariq, S., Singh, R., Jalalvand, F., Paris, C. and Nepal, S. (2024) 'Towards human-AI teaming to mitigate alert fatigue in Security Operations Centres', *ACM Transactions on Internet Technology*, 24(3), Article 12, pp. 1–22. doi:10.1145/3670009.
- Gelman, B., Taoufiq, S., Vörös, T. and Berlin, K. (2023) 'That escalated quickly: An ML framework for alert prioritization', *arXiv preprint*, arXiv:2302.06648.
- Gupta, N., Traore, I. and Quinan, P.M.F. (2019) 'Automated event prioritization for Security Operation Center using deep learning', in *2019 IEEE International Conference on Big Data (Big Data)*, Los Angeles, CA, USA, pp. 5864–5872.
- Ismail, Kurnia, R., Brata, Z.A., Nelistiani, G.A., Heo, S., Kim, H. and Kim, H. (2025) 'Toward robust security orchestration and automated response in Security Operations Centers with a hyper-automation approach using agentic artificial intelligence', *Information*, 16, 365. doi:10.3390/info16050365.
- Karlzén, H. and Sommestad, T. (2023) 'Automatic incident response solutions: A review of proposed solutions' input and output', in *The 18th International Conference on Availability, Reliability and Security (ARES 2023)*, Benevento, Italy, pp. 1–9. doi:10.1145/3600160.3605066.
- Khayat, M., Barka, E., Serhani, M.A., Sallabi, F., Shuaib, K. and Khater, H.M. (2025) 'Empowering Security Operation Center with artificial intelligence and machine learning—A systematic literature review', *IEEE Access*, 13, pp. 19162–19197. doi:10.1109/ACCESS.2025.3532951.
- Kinyua, J. and Awuah, L. (2021) 'AI/ML in Security Orchestration, Automation and Response: Future research directions', *Intelligent Automation & Soft Computing*, 28(2), pp. 527–545. doi:10.32604/iasc.2021.016240.
- Liu, J., Zhang, R., Liu, W., Zhang, Y., Gu, D., Tong, M., Wang, X., Xue, J. and Wang, H. (2022) 'Context2Vector: Accelerating security event triage via context representation learning', *Information and Software Technology*, 146, 106856. doi:10.1016/j.infsof.2022.106856.
- Srinivas, S., Kirk, B., Zendejas, J., Espino, M., Boskovich, M., Bari, A., Dajani, K. and Alzahrani, N. (2025) 'AI-Augmented SOC: A survey of LLMs and agents for security automation', *Journal of Cybersecurity and Privacy*, 5, 95. doi:10.3390/jcp5040095.
- Sworna, Z.T., Babar, M.A. and Sreekumar, A. (2023) 'IRP2API: Automated mapping of cyber security incident response plan to security tools' APIs', in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 546–557. doi:10.1109/SANER56733.2023.00057.

Wang, X., Yang, X., Liang, X., Zhang, X., Zhang, W. and Gong, X. (2024) ‘Combating alert fatigue with AlertPro: Context-aware alert prioritization using reinforcement learning for multi-step attack detection’, *Computers & Security*, 137, 103583. doi:10.1016/j.cose.2023.103583.

Zhang, J., Li, S., Huang, W., Jing, H., Zhang, Q. and Xia, X. (2025) ‘Design and computational modeling of an AI-based automated cybersecurity incident response system’, *IEEE Access*, 13, pp. 154383–154394. doi:10.1109/ACCESS.2025.3603975.